When Does Keeping Agent Sessions in GPU Memory Pay Off?

A Rust Serving Engine and a Replay Study on Real Agent Traces

Nahuel Alejandro Nucera
nahuelnucera990@gmail.com

November 2026

Abstract

AI agents call a language model, run a tool, and call the model again with the whole history attached. Serving engines such as vLLM and SGLang reuse cached prompt prefixes, but under memory pressure they evict the history of agents that are waiting on a tool and recompute it when the agent returns. We built a serving engine in Rust (about 1,700 lines) that keeps agent sessions in GPU memory between turns, shares common prompt prefixes across sessions, and admits new sessions only when the sessions already in memory still fit. We replayed 24 real coding-agent sessions (791 turns) against our engine, vLLM 0.33.0 and SGLang 0.5.23 on the same GPU, model and KV cache size, with two repeat runs per configuration. When the agent's tool call takes 5 seconds, our engine finishes the workload 1.13x faster than the best baseline (1.24x faster than vLLM) and returns the first token about 5x sooner. When the tool call takes 60 seconds, all engines finish within 2% of each other and our engine is slower to the first token. Three pinning policies we tried and pre-registered pass/fail criteria for did not beat the baselines at 60 seconds; we report them with the data.

1 Introduction

A coding agent works in turns. It sends the conversation so far to the model, receives a tool call, runs the tool (a shell command, a file edit), appends the result and sends the conversation again. In the traces we use, the median session has about 60 turns and grows to tens of thousands of tokens. Every turn re-sends everything before it.

Serving engines handle repeated prefixes with a prefix cache: vLLM hashes KV-cache blocks [1], SGLang keeps a radix tree of cached prefixes [2]. Both evict least recently used entries when memory runs out. An agent waiting on a tool is, from the cache's point of view, idle, so its history is a natural eviction candidate. When the agent returns, its history must be computed again. Continuum [4] showed that pinning an agent's KV cache across tool calls, with a time-to-live, can cut job completion time, and a recent paper on stateful inference for multi-agent tool calling [5] reports per-turn speedups from never recomputing history.

We ask a narrower, practical question: on real agent traces and the same hardware as current engines, when does keeping sessions in memory beat the established prefix caches, and when does it not? To answer it we wrote a serving engine with sessions as a first-class concept and compared it against vLLM and SGLang under controlled conditions.

Contributions:

- An open-source serving engine in Rust with per-session KV retention, cross-session prefix sharing, session-aware admission and three pinning policies, verified against vLLM and an FP32 reference (Sections 3, 4).

- A replay benchmark of 24 real agent sessions with matched KV cache size across engines, run inside the server container to avoid network effects, with repeat runs and pre-registered pass/fail criteria (Section 5).
- Results: a 1.13x end-to-end gain over the best baseline and about 5x faster time to first token when tool calls are short; parity when they are long (Section 6).
- Negative results: pinning policies that made things worse, and why (Section 7).

2 Background and related work

Paged KV cache and prefix caching. vLLM stores the KV cache in fixed-size blocks [1]. Version 0.33.0 identifies a block by a hash chained over its tokens and the previous block, reuses matching blocks across requests, and evicts unreferenced blocks in least-recently-used order. Its OpenAI-compatible endpoint accepts a `session_id`, but in this version it only labels KV-cache events; it does not keep or pin anything (we checked `vllm/v1/core/block_pool.py`). SGLang keeps cached prefixes in a radix tree (RadixAttention [2]) and evicts leaves first. Version 0.5.23 offers several eviction policies, including Tail-Optimized LRU (TLRU) [3], which prefers to evict the part of a conversation that cannot affect tail time to first token.

Sessions. SGLang 0.5.23 has a session API (`/open_session`, `/close_session`) through its native `/generate` endpoint; its OpenAI chat endpoint forwards only a `session_id`. A streaming-session mode, off by default, locks a session's KV until the session is closed. We found no time-to-live per turn and no session-aware admission in its schedulers (first-come-first-served by default).

Agent-aware serving. Continuum [4] pins an agent's KV cache across tool calls with a time-to-live computed from reload cost and queueing delay, and schedules whole agent programs first-come-first-served. Norgren [5] keeps a persistent KV cache per agent so that each turn processes only new tokens. Our pinning and admission mechanisms are close to Continuum's; we do not claim them as new. Our contribution is the measurement against current engines on real traces, the open implementation, and the negative results.

3 Design

Sessions. A request may carry a `session_id`. When the request finishes, the engine keeps the session's token sequence and KV blocks. The next request of the same session reuses the longest common token prefix and computes only what follows.

Prefix sharing. Each full 64-token block gets a hash chained from the previous block's hash, so a hash identifies the whole prefix up to the end of its block. An index maps hashes to physical blocks. A new request takes the longest run of indexed blocks matching its tokens, from any session. Blocks are reference counted; a block nobody holds moves to an LRU list but stays in the index until its space is handed out. Indexed or shared blocks are never written: if a session must roll back into the middle of one, the rollback point is moved down to the block boundary and the rest is recomputed. This matters for agents: in our traces all 24 sessions start with the same 5,636 tokens (system prompt and tool definitions). Without sharing, those would be stored 24 times, about 78% of the KV cache.

Pinning. After a turn, a session's KV may be *pinned*: kept out of eviction for a while so that it survives the agent's tool call. We built three policies:

- **Fixed**: pin for 30 seconds after every turn.
- **Adaptive**: the engine learns, per tool name, the gap between a turn that called the tool and the session's next request, and pins for the 90th percentile of that gap times 1.5 plus 2 seconds. A turn the model ends without a tool call is not pinned.
- **Capped**: pin only if the learned wait is no longer than the time needed to recompute a full KV cache (capacity divided by prefill speed, 173,200 tokens / 10,000 tokens per second, about 17 seconds), and only while pinned idle sessions hold at most half the cache.

Admission. A turn of a session already in memory is admitted first. A new session is admitted only if every resident session, counted at its current length times 1.2, plus the new prompt, still fits in the cache. Otherwise it waits. If running requests cannot grow, the most recently admitted one is preempted: its blocks are freed and it is recomputed later.

Steps. Each step processes at most 4,096 new tokens: running requests that generate one token go first, the rest of the budget goes to prompts in chunks. Steps in which every request generates exactly one token use a dedicated decode kernel.

4 Implementation

The engine is 1,709 lines of Rust. Model code (Qwen3) uses candle [6] (a fork at revision 8ac335c) and attention-rs [7] (revision 3c258f0) for paged attention through FlashInfer [8]. Q, K and V projections, and the gate and up projections, run as single stacked matrix multiplications; the residual add and RMSNorm after attention, and SiLU with multiplication, use fused kernels. The server speaks the OpenAI chat-completions API with streaming, renders the model's own chat template, and parses tool calls. Builds and benchmarks run on Modal; the engine runs on one NVIDIA L40S.

Correctness. We checked the engine in four ways. (1) Teacher-forced against vLLM on the same 64 tokens of Qwen3-8B: the engine's top token matches at all 64 steps. (2) Against an FP32 run of the same model in Hugging Face Transformers: mean absolute log-probability error 0.0056 on the reference tokens (vLLM: 0.0043), and the same top token as FP32 at all 64 steps. (3) A session's second turn on a live cache gives the same greedy output as a fresh run over the full history. (4) A request that reuses another request's cached prefix gives the same greedy output as on an empty cache.

5 Methodology

Workload. We use real coding-agent trajectories from SWE-rebench-OpenHands [9] (67,074 sessions collected with OpenHands and Qwen3-Coder-480B; CC-BY-4.0). We sampled 64 sessions with a fixed seed and use the first 24. Each session is replayed turn by turn: the prompt for turn t is every recorded message before the t-th assistant message, with the recorded tool definitions. The engine generates 256 tokens per turn (end-of-sequence ignored, so all engines do the same work). Between turns the client waits a fixed time standing in for the tool call: 5 seconds or 60 seconds.

Table 1: Wall time for all 24 sessions, two runs each (seconds; lower is better). Ratios use means.

Engine	Tool wait 5 s	Tool wait 60 s
vLLM 0.33.0	2342 / 2333	4106 / 4128
SGLang 0.5.23, LRU	2179 / 2154	4069 / 4154
SGLang 0.5.23, TLRU	2121 / 2152	4078 / 4093
This engine (capped)	**1868 / 1908**	4083 / 4011
vs best baseline (SGLang TLRU)	1.13x	1.01x
vs vLLM	1.24x	1.02x

Table 2: Time to first token, median and 90th percentile over all turns, per run (seconds), and recomputed history (% of prompt tokens).

	Tool wait 5 s		Tool wait 60 s	
Engine	TTFT p50 / p90	Recomp.	TTFT p50 / p90	Recomp.
vLLM	26.8, 24.9 / 43.2, 43.6	55.5, 54.9	3.5, 3.8 / 16.6, 17.0	55.0, 54.9
SGLang LRU	29.2, 29.4 / 44.0, 46.8	53.5, 52.7	6.1, 7.1 / 18.4, 19.4	52.6, 51.8
SGLang TLRU	29.3, 25.3 / 45.8, 44.9	53.0, 52.7	6.3, 6.4 / 18.1, 18.8	53.1, 52.9
This engine	**5.1, 6.3 / 26.0, 28.8**	**18.6, 23.1**	13.8, 11.5 / 23.0, 21.1	56.5, 54.6

A session stops after its prompt passes 30,000 tokens. All 24 sessions run concurrently. Our engine receives a `session_id` per session; vLLM and SGLang rely on their prefix caches.

Setup. Qwen3-8B (revision b968826), bfloat16, one L40S per engine. Every engine gets the same KV cache size: vLLM 0.33.0 allocates 173,200 tokens at `gpu_memory_utilization` 0.92; we set our engine to 2,706 blocks of 64 tokens (173,184) and SGLang 0.5.23 to `--max-total-tokens 173200 --mem-fraction-static 0.92` (with its default memory fraction it allocated only 158,237). SGLang runs with LRU eviction and, separately, with TLRU (threshold 4,096 tokens, as in its documentation; next-prompt estimate 434 tokens, the median per-turn prompt growth we measured on these traces). The replay client runs inside the server's container and connects to localhost: through Modal's proxy, streamed chunks arrived in bursts for every engine, which made time-to-first-token measurements unusable.

Metrics. *Wall time*: time to finish all 24 sessions (primary). *Recomputed history*: tokens the engine had already seen on the previous turn of the same session but did not serve from cache, as a share of all prompt tokens. *Time to first token* (TTFT): from sending a turn to receiving its first streamed token, median and 90th percentile over all turns.

Repeats and criteria. Every configuration ran twice. Identical runs of our engine differed by up to 7.7% in wall time; vLLM's by 0.5%. Before seeing the results of each memory-policy change we wrote down, with a timestamp, the result that would count as a pass and the decision we would take otherwise.

Table 3: Memory policies tried (mean wall time in seconds, two runs each; vLLM: 2338 at 5 s, 4117 at 60 s).

Policy	Commit	5 s	60 s
Fixed 30 s pin	92477a8	2060	5342
Adaptive pin per tool	92477a8	not run	6575
Capped pin	96600a3	2097	4261
Capped pin + prefix sharing	38c91d3	2863	stopped
Capped pin + prefix sharing, fixed	1c92959	**1888**	**4047**

6 Results

Short tool calls. With 5-second tool calls (Table 1), our engine finishes 1.13x faster than the best baseline and 1.24x faster than vLLM, in both runs. The baselines recompute more than half of all prompt tokens as history they had already seen; our engine recomputes 19–23%. The median turn gets its first token about 5x sooner (Table 2). Among the baselines, SGLang is 8–9% faster than vLLM, and TLRU changes SGLang's results by about 1%.

Long tool calls. With 60-second tool calls, all engines finish within 2% of each other, within the run-to-run spread of our engine. Our engine recomputes about as much history as the baselines, and its first token is slower: median 11.5–13.8 seconds versus 3.5–3.8 for vLLM and 6.1–7.1 for SGLang. With long waits, few agents need the GPU at the same moment, so queues are short for every engine. Our likely explanation for the slower first token, which we did not measure separately, is that our admission rule makes sessions wait where the baselines' first-come-first-served queues do not.

Prefix sharing. Adding cross-session prefix sharing to the capped policy moved our engine at 60 seconds from 3–4% slower than vLLM to parity, and at 5 seconds from 2097 to 1888 seconds mean wall time (Section 7).

Raw speed. Without any reuse, our engine generates tokens 15–21% slower than vLLM (25.1 / 27.8 / 29.7 ms per token at 1 / 8 / 24 concurrent requests, versus 21.8 / 23.5 / 24.6 ms; SGLang 21.5 / 24.5 / 25.3 ms). It processes a single long prompt at about vLLM's speed (11,862 versus 12,419 tokens per second for 3,000 tokens) but does not speed up when several prompts arrive together (11,791 versus 14,808 for eight). A profile of prompt processing showed RMSNorm costing more GPU time per token than all matrix multiplications before we switched to a fused kernel. Our wins come from doing less work, not from faster kernels.

7 What did not work

Table 3 lists every memory policy we measured.

Fixed pin. At 60 seconds, a 30-second pin expires before the agent returns. The session then counts as new, waits behind the admission rule, and its trimmed cache must be recomputed: 85% of history recomputed, 1.3x slower than vLLM.

Adaptive pin. Learning the wait per tool fixed the expiry problem (53% recomputed, about vLLM's level) but made wall time worse: 1.6x slower than vLLM and a 90th-percentile TTFT of over four minutes. By the policy's formula, a 60-second wait leads to pins of roughly 90 seconds; our reading, not measured directly, is that these idle pinned sessions blocked new ones and left the GPU with too little work. Our pre-registered criterion counted this policy as a pass because we had written "wall time *or* recomputed history"; we report it as worse on the metric that matters, and since then name one primary metric per experiment.

Capped pin. Pinning only when waiting is cheaper than recomputing, and capping pinned memory, came within 3–4% of vLLM at 60 seconds but still failed the criterion.

Prefix sharing, first version. Our first integration released whole sessions to the shared cache under pressure; a session without blocks then counted as new and waited behind the admission rule, giving a 90th-percentile TTFT above 100 seconds and failing at 5 seconds. We declared a single fix in advance (a returning session skips the new-session gate even if its blocks were released), kept the criterion unchanged, and reran once (last row).

Fused RMSNorm. The fused RMSNorm kernel from attention-rs brought prompt processing to vLLM's speed for single prompts but moved our log-probabilities further from FP32 (mean error 0.0071 versus 0.0053) and changed the top token at one of 64 steps. We use candle's fused kernel instead (error 0.0056, no top-token change, 11–14% faster prefill than unfused).

Measurement pitfalls. Two harness problems produced misleading numbers before we caught them: time to first token measured through a proxy (chunks delivered in bursts) and HTTP connections reused across a 5-second tool wait, which raced the server's 5-second keep-alive timeout and failed requests for vLLM and our engine alike.

8 Limitations

One model (Qwen3-8B), one GPU type (L40S), one load level (24 concurrent sessions) and two tool-wait values. Tool time is a fixed wait because the traces do not record real tool durations; real agents mix short and long tool calls. Replayed turns ignore the model's actual output: the next turn is always the recorded one. vLLM and SGLang reject prompts over 32,768 tokens and our engine does not, so it served 2–3 more turns of one session than the baselines. Two runs per configuration bound but do not characterize the variance of our engine, whose runs differed by up to 7.7%.

9 Conclusion

Keeping agent sessions in GPU memory helps when agents come back quickly: 1.13x faster end to end than the best current engine and about 5x faster to first token on real coding-agent traces. When agents are away for a minute, the advantage disappears and none of the three pinning policies we tried beat the established prefix caches; prefix sharing across sessions was what brought our engine back to parity. Whether to keep a session's cache depends on how long the agent will be away relative to the cost of recomputing its history, and a policy that decides this well for mixed tool times remains open. The code, data and the scripts that produce every number in this paper are available under the MIT license.

References

[1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. arXiv:2309.06180.

[2] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, Y. Sheng. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104, 2023.

[3] W. Zhang, Y. Li, C. C. Moallemi, T. Peng. Tail-Optimized Caching for LLM Inference. arXiv:2510.15152, 2025.

[4] H. Li, R. He, Q. Mang, Q. Zhang, H. Mao, X. Chen, H. Zhou, H. Zhang, A. Cheung, J. Gonzalez, I. Stoica. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv:2511.02230.

[5] V. Norgren. Stateful Inference for Low-Latency Multi-Agent Tool Calling. arXiv:2605.26289, 2026.

[6] Hugging Face. candle: a minimalist ML framework for Rust. https://github.com/huggingface/candle; fork used: https://github.com/guoqingbao/candle.

[7] G. Bao. attention.rs. https://github.com/guoqingbao/attention.rs.

[8] FlashInfer: Kernel Library for LLM Serving. https://github.com/flashinfer-ai/flashinfer.

[9] Nebius. SWE-rebench-OpenHands-Trajectories. https://huggingface.co/datasets/nebius/SWE-rebench-openhands-trajectories.